Regime-Gated Residual Mixture-of-Experts for Cross-Sectional Volatility Forecasting

Junyi Ye
School of Computing
Montclair State University
Montclair, New Jersey, USA
yej@montclair.edu

Gargi Vijay Borde
School of Computing
Montclair State University
Montclair, New Jersey, USA
bordeg1@montclair.edu

Abstract

Financial volatility is regime dependent, yet incorporating regime information into neural networks can also destabilize training. This paper asks how such information should enter a neural cross-sectional volatility forecasting model. We study five-day realized-volatility forecasts for 1,027 U.S. equities using a rolling walk-forward evaluation framework in which information, model capacity, hyperparameter tuning, and random seeds are matched across architectures. We propose RG-ResMoE, a regime-gated residual mixture-of-experts architecture in which regime information is used only for expert routing rather than for direct forecasting. RG-ResMoE consistently outperforms a capacity-matched MLP in forecasting accuracy, Value-at-Risk calibration, and training stability in the main U.S. study. Similar gains are observed on an independent Japanese panel. Ablations show that these gains come primarily from the residual design. The integration pathway is decisive: appending the same regime variables directly to the forecasting input degrades both predictive performance and training stability, whereas restricting them to the routing gate keeps accuracy comparable and training stable. Hard routing consistently underperforms soft routing. The results suggest that, in compact neural volatility forecasting models, the primary value of mixture-of-experts models lies less in increasing model capacity than in controlling how nonstationary regime information influences prediction.

Code Availability: https://github.com/JunyiYe/rg-resmoe

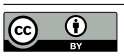
CCS Concepts

• **Computing methodologies** → **Neural networks**; *Ensemble methods*; • **Mathematics of computing** → **Time series analysis**; • **Applied computing** → *Economics*.

Keywords

mixture of experts, residual learning, gated routing, volatility forecasting, training stability

ACM Reference Format:
Junyi Ye and Gargi Vijay Borde. 2026. Regime-Gated Residual Mixture-of-Experts for Cross-Sectional Volatility Forecasting. In *7th ACM International Conference on AI in Finance (ICAIF '26), November 14–17, 2026, Milan, Italy*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3858063.3858798



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICAIF '26, Milan, Italy
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-3087-0/2026/11
https://doi.org/10.1145/3858063.3858798

1 Introduction

Financial volatility forecasting is central to portfolio allocation, derivative pricing, and risk management. Deep neural networks have substantially improved volatility prediction by learning nonlinear relationships from large cross-sectional equity panels. However, financial markets are inherently nonstationary: the relationship between historical observations and future volatility changes across periods of calm, crisis, and recovery. Regime information is therefore a natural source of additional context for neural forecasting models. The central question, however, is not whether market regimes matter, but how such information should be incorporated into the forecasting process.

Existing approaches incorporate regime information in different ways. Some models use regime-related variables as forecasting inputs [21], while others adapt model structures to market changes [14]. Regime-switching models capture state-dependent dynamics through distinct parameters [8, 9], while mixture-of-experts (MoE) architectures learn input-dependent expert routing [10]. However, evaluating these approaches can involve confounding factors such as regime information, model capacity, and architectural design. Comparing an MoE with a smaller single-network baseline makes it difficult to distinguish architectural gains from those due to additional parameters. Similarly, introducing regime variables alongside architectural changes obscures whether improvements arise from the information itself or how it is incorporated. These distinctions matter in financial forecasting, where predictive gains are often small and optimization can be sensitive to initialization.

We disentangle these factors through controlled comparisons of model capacity, regime information, and expert architecture. We use capacity-matched MLP baselines, compare direct input concatenation with expert gating, and examine residual learning and routing through ablations. All models follow the same rolling walk-forward evaluation and hyperparameter selection protocol. We propose *RG-ResMoE*, which combines a frozen base forecaster with small, zero-initialized residual experts. A soft gate uses stock features and regime variables to weight expert corrections, while the forecasting networks receive only stock features. This design separates the effects of additional information, model capacity, and architectural choices.

Experiments on 1,027 U.S. equities from 2018 to 2025 reveal three findings. First, directly appending regime variables reduces forecasting accuracy and increases training instability, whereas gated residual routing preserves both. Second, continuous soft routing outperforms hard routing based on learned assignments or predefined market partitions. Third, RG-ResMoE improves Value-at-Risk

calibration and achieves its largest forecasting gains during high-volatility periods. Similar improvements in training stability are observed on an independent Japanese equity panel.

More broadly, our results suggest that the primary value of regime-aware MoE models in compact financial forecasting systems lies less in increasing model capacity than in controlling how nonstationary market information influences the prediction. Taken together, these findings establish a simple design principle: in compact neural volatility forecasters, regime information can be incorporated safely when it guides residual routing, whereas treating it as ordinary predictive input degrades accuracy and training stability. We believe this principle extends beyond the proposed RG-ResMoE architecture and provides guidance for incorporating market state information into future neural forecasting models.

2 Data and Task

2.1 Equity panels

Our main experiments use a U.S. equity panel, with a Japanese panel reserved for cross-market replication. The U.S. panel contains 1,027 common stocks from Yahoo Finance, observed from December 2015 to November 2025. The universe spans all eleven sectors and large-, mid-, and small-cap stocks. Among these stocks, 85% are current S&P 1500 constituents. The panel is restricted to stocks that remain listed through the end of the sample and have at least six years of price history. Among them, 92% cover the full ten-year period.

The replication panel contains 1,552 Japanese stocks from Yahoo Finance over the same period and covers the TSE Prime segment. It is constructed using the same data-processing pipeline and serves as a second-market replication of the main U.S. results.

2.2 Inputs and regime state variables

For each stock i on day t, the **stock-level input** $x_{i,t}$ contains sixteen features computed from the stock's own price history. These include realized volatility over the trailing 5, 20, and 60 days, cumulative returns over the trailing 5 and 20 days, the 14-day relative strength index (RSI), and the ten most recent daily log returns. For each walk-forward window, normalization parameters are estimated from the training data and then applied to the validation and test data.

The **regime state variable** $z_{i,t}$ consists of two volatility-based indicators: market volatility and idiosyncratic volatility. Market volatility is defined as the 20-day rolling volatility of the equal-weighted market return $r_{m,t}$ and is shared by all stocks on a given trading day. Idiosyncratic volatility is defined as the 20-day rolling volatility of the residual return $r_{i,t} - \beta_{i,t} r_{m,t}$, where $r_{i,t}$ is the return of stock i on day t, $r_{m,t}$ is the market return, and $\beta_{i,t}$ is estimated from the preceding 120 trading days. Market volatility captures the overall level of systematic uncertainty, whereas idiosyncratic volatility measures stock-specific uncertainty after removing broad market movements. Together, these variables provide complementary information about the prevailing market regime. Across experiments, both state variables are fixed, and only their point of integration into the model is varied.

2.3 Task

The task is to predict cross-sectional stock volatility. For stock i on day t, the target is the annualized five-day forward realized volatility

$$y_{i,t} = \sqrt{252}\ \mathrm{Std}\left(r_{i,t+1}, \ldots, r_{i,t+5}\right), \tag{1}$$

where $r_{i,t+k}$ denotes the daily log return of stock i on day $t+k$. Models generate predictions for all stocks on each trading day.

Table 1: Rolling walk-forward evaluation protocol.

Window	Development period	Test period
1	2016-04 – 2018-03	2018-04 – 2018-06
2	2016-07 – 2018-06	2018-07 – 2018-09
⋮	⋮	⋮
30	2023-08 – 2025-07	2025-08 – 2025-10

Evaluation follows the rolling walk-forward protocol illustrated in Table 1. Each evaluation window uses a 504-trading-day (two years) development period, with the first 85% used for model training and the final 15% used for validation, followed by a 63-trading-day (one quarter) test period. After each evaluation period, the full window advances by 63 trading days. This procedure produces 30 non-overlapping test windows from April 2018 to October 2025 and approximately 1.9 million out-of-sample forecasts. The evaluation period spans diverse market regimes, including the COVID-19 market crash in 2020, the subsequent recovery in 2021, the broad bear market of 2022, and comparatively calmer periods before and afterward.

3 Model Architecture

All neural network models are built from the same two-hidden-layer MLP block, which serves as the common forecasting component across all architectures. The variants differ in three aspects: the use of regime state variables, the routing mechanism, and whether experts predict full volatility forecasts or residual corrections to a shared base forecast.

3.1 Shared MLP block

To keep the forecasting component consistent across models, every neural network is constructed from the same two-hidden-layer MLP block. Given an input vector q, the block is defined as

$$\begin{aligned} h_1 &= \phi(W_1 q + b_1), \\ h_2 &= \phi(W_2 h_1 + b_2), \\ \mathrm{Block}(q;\theta) &= W_3 h_2 + b_3. \end{aligned} \tag{2}$$

Here, ϕ denotes the GELU activation. Both hidden layers have width H, and dropout with rate 0.1 is applied after each hidden layer during training. Let x denote the stock-level feature vector and z denote the vector of regime state variables. The MLP baselines receive either the stock features alone, $q = x$, or the concatenated stock and state inputs, $q = (x, z)$. In MoE and RG-ResMoE, each forecasting block receives only x, while z can affect the prediction only through the gate.

3.2 Standard MoE

A standard mixture-of-experts (MoE) model combines the predictions of multiple experts, each implemented as an MLP block, through a learned gate. Intuitively, different experts can specialize

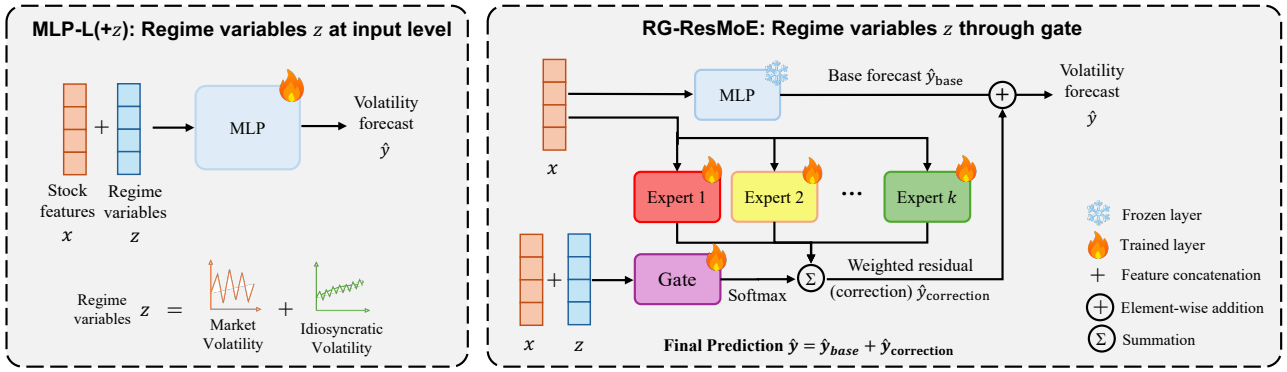


Figure 1: Architectures of the baseline MLP-L(+z) (left) and proposed RG-ResMoE (right).

in different market conditions or stock-level patterns, while the gate determines how much each expert contributes to the final prediction. Given the stock-level feature vector x, each expert produces a volatility forecast, and the gate combines these forecasts as

$$\begin{aligned} e_k(x) &= \text{Block}(x;\theta_k), \\ \pi &= \text{softmax}\big(g(u)\big), \\ \hat{y} &= \sum_{k=1}^{K} \pi_k e_k(x). \end{aligned} \tag{3}$$

Here, $e_k(x)$ is the volatility forecast produced by the kth MLP expert, θ_k denotes its parameters, g is the gate network, and π_k is the weight assigned to expert k. Each expert receives x, while the gate receives both the stock-level features and the volatility state variables, so $u = (x, z)$. The gate therefore provides input-dependent conditional weighting. All experts and the gate are trained jointly.

3.3 Regime-gated Residual MoE (RG-ResMoE)

Unlike the standard MoE, in which each expert predicts the full volatility level, RG-ResMoE decomposes the prediction into a base network and expert-specific residual corrections, as illustrated in Figure 1. The base network captures common forecasting patterns, while the experts model conditional deviations from the base prediction. This decomposition allows the experts to focus on specialized corrections rather than relearning the full forecasting function.

Given the stock-level feature vector x, the base network produces

$$\hat{y}_{\text{base}} = \text{Block}(x;\theta_b), \tag{4}$$

where $\hat{y}_{\text{base}}$ is the base forecast. The kth expert produces a residual correction,

$$r_k(x) = \text{Block}(x;\theta_k). \tag{5}$$

The gate then combines the expert corrections as

$$\begin{aligned} \pi &= \text{softmax}\big(g(u)\big), \\ \hat{y}_{\text{correction}} &= \sum_{k=1}^{K} \pi_k r_k(x), \\ \hat{y} &= \hat{y}_{\text{base}} + \hat{y}_{\text{correction}}. \end{aligned} \tag{6}$$

Here, θ_b and θ_k denote the parameters of the base network and the kth expert, respectively, and π_k is the weight assigned to expert k. The aggregated residual correction $\hat{y}_{\text{correction}}$ is the soft-weighted sum of the expert outputs. The base network and all experts receive x, while the gate receives $u = (x, z)$.

Table 2: Models and architectural variants considered in the experiments. +z denotes direct input integration of regime variables, while -z removes the regime pathway from RG-ResMoE.

Category	Models
Main models	RG-ResMoE
Regime state variants	RG-ResMoE (-z); Ridge (+z); MLP-S (+z); MLP-L (+z)
Soft routing	Learned soft gate
Hard routing	Learned top-1 gate; volatility-quantile assignment; market×idiosyncratic volatility split; GICS sectors
Architecture variants	Standard MoE; random initialization; RG-ResMoE-K2; RG-ResMoE-K6
Baselines	Persistence; GARCH; HAR; Ridge; MLP-S; MLP-L

Training proceeds in two stages. The base network is trained first and then frozen. The $K = 4$ correction experts and the gate are then trained jointly by minimizing

$$\mathcal{L} = \text{MSE}(\hat{y}, y) + \alpha \overline{\left(\sum_{k=1}^{K} \pi_k r_k(x)\right)^2} + \lambda_{\text{LB}} \sum_{k=1}^{K} \left(\bar{\pi}_k - \frac{1}{K}\right)^2. \tag{7}$$

Here, the bar denotes an average over the full training batch. The first term is the prediction loss. The second term penalizes the magnitude of the aggregate residual correction, encouraging the final prediction to remain close to the frozen base forecast $\hat{y}_{\text{base}}(x)$. The coefficient α controls the strength of this shrinkage. The third term is a load-balancing penalty that discourages the routing weights from concentrating on a small number of experts. The coefficient λ_{LB} controls how strongly the routing weights are encouraged toward the uniform allocation $1/K$. Together, these regularizers keep the expert corrections complementary to the base forecast and reduce the risk of routing collapse. Each correction expert uses a zero-initialized final layer, so $r_k(x) = 0$ at initialization and the model initially reproduces the base forecast $\hat{y}_{\text{base}}(x)$.

3.4 Routing strategies

The routing mechanism determines how expert predictions are selected and combined. We consider both soft and hard routing. In soft routing, a gate network produces a probability distribution over the experts, $\pi = \mathrm{softmax}(g(u))$, and the final prediction is formed as a weighted combination of all expert outputs. This allows multiple experts to contribute to each forecast, with their weights varying continuously across observations.

In hard routing, each observation is assigned to a single expert. The selected expert may be determined either by a learned top-1 gate or by predefined rules based on market characteristics, such as volatility regimes or sector membership. The prediction is produced solely by the selected expert, without averaging across experts.

3.5 Comparison set

Table 2 summarizes the models evaluated in this study. The comparison includes classical statistical baselines, MLP architectures of different capacities, MoE and the proposed RG-ResMoE family. Beyond the main RG-ResMoE model, the variants isolate three design dimensions: the use of regime state variables, the routing mechanism, and the RG-ResMoE architecture itself.

4 Evaluation Protocol

4.1 Training and hyperparameter tuning

All neural network models use the same training procedure: full-batch Adam optimization of mean-squared error (MSE), early stopping based on validation loss, and restoration of the best checkpoint. Each reported configuration is trained with 30 random seeds, while hyperparameter sweeps use three seeds. This design allows us to evaluate both average performance and sensitivity to random initialization under a fixed training budget.

Hyperparameters are selected using validation information coefficient (IC), defined as the cross-sectional correlation between predicted and realized volatility. Candidate configurations are filtered using three robustness criteria. The selected configuration (i) lies in the interior of the search grid rather than at its boundary, (ii) achieves a mean validation IC exceeding that of the runner-up by more than its across-seed standard deviation, and (iii) has a validation MSE within one standard deviation of the lowest MSE observed in the sweep. The selected RG-ResMoE configuration trains the base network with a learning rate of 0.03 and the experts and gate jointly with a learning rate of 0.01, $\alpha = 1.0$, $\lambda_{\mathrm{LB}} = 0.5$, and an early-stopping patience of 50 epochs.

4.2 Baselines

The comparison set includes three classical forecasting baselines: 20-day persistence, pooled HAR [4], and per-stock GARCH(1,1) [3]. Ridge provides a linear baseline using the same stock-level features as the neural networks. MLP-S (**small**) uses a single MLP block with hidden width H=16, matching an individual block in RG-ResMoE. MLP-L (**large**) uses the same block structure with a larger hidden width H=44 to match the overall forecasting capacity of the full RG-ResMoE model. To distinguish the effect of using the regime variables as predictor inputs from their use in expert routing, we additionally evaluate Ridge(+z), MLP-S(+z), and MLP-L(+z). These variants append the two regime state variables to the ordinary model inputs and are re-tuned from scratch.

4.3 Evaluation metrics

Our primary accuracy metrics are IC, RMSE, and out-of-sample R^2. We additionally report ICIR, QLIKE, VaR calibration, and the frequency of collapsed seeds to assess ranking consistency, volatility calibration, and training stability.

Ranking performance: Daily cross-sectional Spearman information coefficient (IC) between predicted and realized volatility, averaged over test days, and information coefficient information ratio (ICIR), defined as the mean daily IC divided by its time-series standard deviation.

Level accuracy: Root mean-squared error (RMSE) and out-of-sample R^2, pooled across all stock-days and computed on the annualized volatility scale.

Quasi-likelihood loss (QLIKE): The volatility loss

$$\mathrm{QLIKE}_{i,t} = \frac{\sigma_{i,t}^2}{\hat{\sigma}_{i,t}^2} - \ln\left(\frac{\sigma_{i,t}^2}{\hat{\sigma}_{i,t}^2}\right) - 1, \tag{8}$$

averaged across all stock-days [17]. Lower values indicate better variance forecasts.

Value at Risk (VaR) coverage: The fraction of tickers for which the null of correct unconditional coverage is rejected by Kupiec's test [13] at the 5% significance level. We evaluate both 5% and 1% VaR, using Student-t residual distributions estimated from the corresponding training window.

Training stability: The number of collapsed seeds, defined using a prespecified diagnostic threshold as runs whose mean test QLIKE exceeds 2.0.

4.4 Statistical testing

Pairwise Diebold–Mariano-style tests [5] assess whether the forecast performance of two competing models differs significantly. For each test day, we compute the difference in the selected metric between models A and B using predictions on the same stocks, then average this difference across the S random seeds:

$$d_t = \frac{1}{S}\sum_{s=1}^{S}\left[X_{A,s}(t) - X_{B,s}(t)\right]. \tag{9}$$

The resulting series of approximately 1,900 daily differences is tested using Newey–West standard errors [16] with four lags to account for dependence induced by the overlapping five-day targets. We report the resulting p-values alongside the model performance metrics.

5 Main Results

Table 3 summarizes the main forecasting results. Across all pooled models, RG-ResMoE achieves the best overall forecast performance. It obtains the highest IC, lowest RMSE, highest out-of-sample R^2, highest ICIR, and lowest QLIKE. Most differences relative to the competing neural networks are statistically significant.

Compared with the classical baselines, all pooled neural network models improve forecast accuracy substantially, demonstrating the benefit of learning shared representations across stocks. Within the

Table 3: Main forecasting results on the U.S. equity panel. Persistence and GARCH are estimated per stock. All other models are pooled across stocks. Neural network results report mean ± standard deviation over 30 seeds. Shading indicates statistically significant paired differences from RG-ResMoE: $p<0.001$, $p<0.01$, and $p<0.05$.

Training	Model	Forecast performance					Stability
		IC ↑	RMSE ↓	R^2 ↑	ICIR ↑	QLIKE ↓	Collapse ↓
Per-stock	Persistence	0.5018	0.2570	0.121	5.18	0.799	—
	GARCH(1,1)	0.5200	0.2642	0.070	5.48	1.120	—
Pooled	HAR	0.5315	0.2347	0.266	5.93	0.749	—
	Ridge	0.5266	0.2336	0.273	5.75	0.760	—
	MLP-S	0.5393 ± 0.0034	0.2323 ± 0.0011	0.280 ± 0.007	6.00 ± 0.15	74.3 ± 395.1	2/30
	MLP-L	0.5421 ± 0.0033	0.2320 ± 0.0009	0.282 ± 0.005	6.08 ± 0.18	2.23 ± 5.15	3/30
	Standard MoE	0.5413 ± 0.0027	0.2357 ± 0.0049	0.259 ± 0.031	6.07 ± 0.14	8,775 ± 18,277	24/30
	RG-ResMoE (ours)	**0.5469** ± 0.0012	**0.2304** ± 0.0013	**0.292** ± 0.008	**6.14** ± 0.06	**0.735** ± 0.017	**0/30**

Table 4: Effect of regime-information integration pathway.

Model	Pathway	IC ↑	ΔIC ↑	Collapse ↓
Ridge	—	0.5266	ref.	—
Ridge (+z)	input	0.5243	−0.0023	—
MLP-S	—	0.5393 ± 0.0034	ref.	2/30
MLP-S (+z)	input	0.5383 ± 0.0041	−0.0011	22/30
MLP-L	—	0.5421 ± 0.0033	ref.	3/30
MLP-L (+z)	input	0.5378 ± 0.0035	−0.0043	30/30
RG-ResMoE (-z)	—	0.5466 ± 0.0011	ref.	1/30
RG-ResMoE	gate	**0.5469** ± 0.0012	**+0.0004**	**0/30**

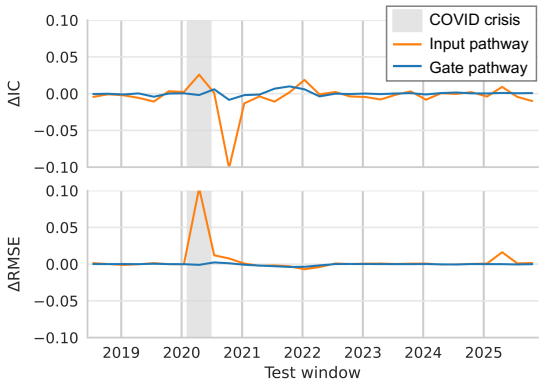


Figure 2: Per-window comparison of input and gate integration across the 30 walk-forward windows. Lines show seed-averaged paired differences within each evaluation window. Positive values indicate improvement for ΔIC and deterioration for ΔRMSE.

pooled models, however, simply increasing model capacity provides only modest gains. Although MLP-L has approximately the same overall capacity as RG-ResMoE, it remains consistently worse across every forecast metric. This indicates that the improvement comes primarily from the residual expert architecture rather than from additional parameters.

The optimization results further support this conclusion. Standard MoE shows frequent optimization failures, collapsing in 24 of the 30 random seeds, whereas RG-ResMoE completes all runs successfully without a single collapse. The **residual decomposition** improves not only forecast accuracy but also training stability.

6 Where the Gain Comes From

6.1 Where should regime information enter the model?

RG-ResMoE uses the two regime state variables only for expert routing, rather than treating them as ordinary forecasting inputs. This design raises a natural question: does the improvement come from the regime information itself, or from how that information is incorporated into the model?

Input integration versus gate-based integration. Table 4 compares two ways of using the same state variables. In the input pathway, the state variables are appended to the stock-level features used by Ridge, MLP-S, and MLP-L, allowing them to directly influence the forecast. In the gate pathway, the forecasting networks continue to use only the stock-level features, while the state variables are used exclusively by the RG-ResMoE gate to determine the mixture of residual experts.

The two integration strategies lead to markedly different outcomes. Appending the state variables to the forecasting inputs consistently reduces IC and substantially increases training instability for the MLP baselines. In contrast, incorporating the same variables through expert routing maintains comparable predictive accuracy while preserving training stability. These results suggest that routing provides a more stable pathway for incorporating regime information than direct input concatenation. Gate-based integration allows the state variables to modulate the expert weights without directly shifting the forecast produced by the underlying forecasting networks.

Behavior across market regimes. The aggregate results in Table 4 may conceal variation across market conditions. Figure 2 therefore reports window-level differences between models with

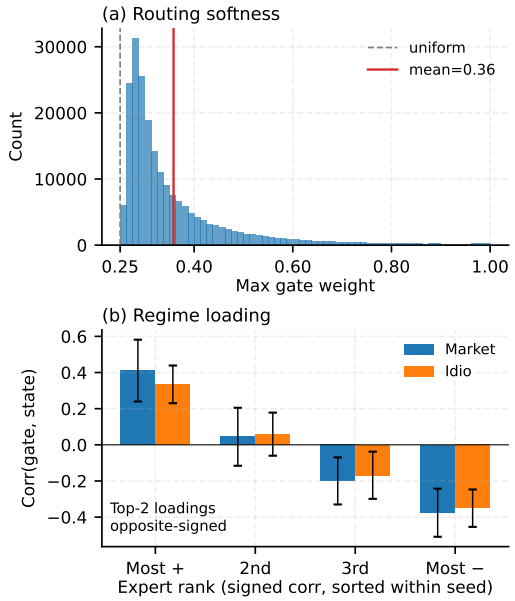


Figure 3: Gate behavior of RG-ResMoE across 30 random seeds. (a) Distribution of the maximum routing weight assigned to any expert for each observation. Uniform routing corresponds to 0.25 and hard routing to 1. (b) Correlations between expert weights and the two regime variables after within-seed sorting. Whiskers denote across-seed standard deviations.

and without regime state variables. For the input pathway, the difference is computed as MLP-L(+z) minus MLP-L; for the gate pathway, it is computed as RG-ResMoE minus RG-ResMoE(-z). Positive values favor the model with regime variables for IC, whereas negative values favor it for RMSE. Positive values indicate improvement for ΔIC and deterioration for ΔRMSE.

The largest difference between the two pathways appears around the transition from the COVID shock to the subsequent market recovery. During the stress period, the input pathway briefly improves IC, but this gain reverses during the recovery phase. RMSE also increases substantially during the stress period before moving back toward zero.

The gate pathway exhibits much smaller changes over the same period and does not show the pronounced IC reversal observed under direct input integration. This comparison suggests that the effect of using regime variables as forecasting inputs is highly dependent on market conditions. Using the same information through the gate instead produces a more stable response during major regime transitions.

6.2 What does the gate learn?

The learned routing remains broadly distributed across experts rather than collapsing to a single component (Figure 3a). The average maximum gate weight remains close to uniform routing, and only a small fraction of observations are assigned almost entirely to one expert. At the same time, expert weights show substantial sensitivity to both market and idiosyncratic volatility, with the most strongly loaded experts typically exhibiting opposite-signed correlations (Figure 3b). Together, these patterns indicate that the gate performs continuous state-dependent reweighting rather than imposing discrete regime assignments.

Table 5: Comparison of soft and hard routing. All variants use the same frozen base and four residual experts. ΔIC and p-values are reported relative to RG-ResMoE.

Variant	Routing	IC ↑	ΔIC ↑	p	Collapse ↓
RG-ResMoE	soft	**0.5469 ± 0.0012**	ref.		**0/30**
learned top-1	hard	0.5448 ± 0.0018	−0.0021	$< 10^{-4}$	1/30
volatility quantiles	hard	0.5447 ± 0.0015	−0.0022	$< 10^{-4}$	1/30
GICS sectors	hard	0.5445 ± 0.0016	−0.0024	$< 10^{-4}$	1/30
market × idio split	hard	0.5435 ± 0.0016	−0.0034	$< 10^{-4}$	1/30

Table 6: Ablations on expert count and residual design. ΔIC is relative to the main RG-ResMoE model.

Variant	IC ↑	ΔIC ↑	Collapse ↓
RG-ResMoE-K4 (main)	**0.5469 ± 0.0012**	ref.	**0/30**
RG-ResMoE-K2	0.5466 ± 0.0014	−0.0004	0/30
RG-ResMoE-K6	0.5467 ± 0.0012	−0.0002	3/30
RG-ResMoE (random init)	0.5430 ± 0.0021	−0.0039	13/30
Standard MoE	0.5413 ± 0.0027	−0.0056	24/30

6.3 Is soft routing better than hard routing?

The previous analysis shows that the gate learns a continuous redistribution of expert weights. An immediate question is whether this continuous routing is necessary, or whether comparable performance can be achieved using discrete expert assignments.

Table 5 compares the learned soft gate with four hard-routing alternatives while keeping the residual architecture and the number of experts fixed. The hard variants include a learned top-1 gate, volatility quantile assignment, GICS sectors, and a market-volatility × idiosyncratic-volatility split. All hard-routing variants underperform the soft gate. Their IC ranges from 0.5435 to 0.5448, corresponding to reductions of 0.0021–0.0034 relative to RG-ResMoE, with every comparison significant at $p < 10^{-4}$. The performance degradation is consistent across all four routing strategies rather than being attributable to a particular hand-crafted partition. These results indicate that the improvement does not arise from a better definition of market regimes. Instead, allowing multiple experts to contribute simultaneously is consistently more effective than assigning each observation to a single expert.

6.4 How many experts are needed?

Two aspects of the expert architecture are examined in Table 6: the number of experts and the residual parameterization. To isolate these effects, the expert-count variants maintain approximately the same overall model capacity by widening individual experts when fewer are used and narrowing them when more experts are introduced.

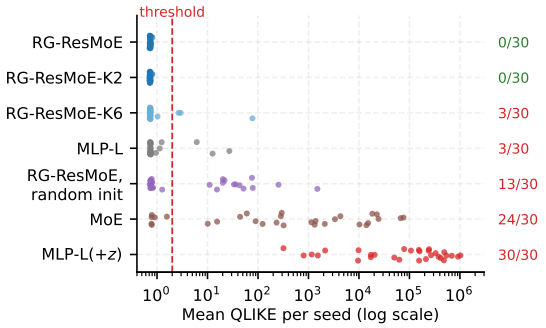


Figure 4: Training stability across random initializations. Each point shows the mean test QLIKE for one training seed. The dashed line marks the collapse threshold (QLIKE = 2), and the numbers on the right report the number of collapsed runs.

The number of experts has only a modest effect on overall forecasting accuracy. Reducing the architecture from four experts to two lowers IC by only 0.0004, indicating that most of the benefit is retained with a smaller mixture. Increasing to six experts provides no further improvement, while reducing stability. Although the average IC remains similar to the four-expert model, three of thirty runs collapse and QLIKE becomes considerably more variable.

The residual parameterization is substantially more important. Replacing the zero-initialized residual layer with random initialization lowers IC and increases the number of collapsed runs from 0 to 13. Removing the frozen base forecast entirely has an even larger effect. The resulting standard MoE exhibits the lowest IC among the neural mixture models and collapses in 24 of 30 runs.

These results indicate that the performance gain does not arise from a large ensemble of experts. Instead, the principal contribution comes from the residual architecture, which anchors expert learning around a shared base forecast while allowing gated residual corrections to model state-dependent deviations.

7 Practical Evaluation

Training reliability. Figure 4 summarizes training reliability. RG-ResMoE and RG-ResMoE-K2 complete all 30 runs without collapse, while RG-ResMoE-K6 and MLP-L each collapse in 3 of 30 runs. Random initialization and standard MoE are substantially less stable, with 13 and 24 collapsed runs, respectively, confirming that the residual design and frozen base are the primary sources of robustness.

Risk calibration. Besides producing accurate forecasts, a volatility model should also provide well-calibrated estimates of downside risk. We therefore convert each five-day volatility forecast into one-day Value-at-Risk (VaR) thresholds, which estimate the loss level expected to be exceeded only with a specified probability (e.g., 5% or 1%).

Table 7 reports the fraction of tickers rejected by Kupiec's coverage test, where lower values indicate better calibrated VaR estimates. RG-ResMoE consistently achieves the lowest rejection rates across nearly all settings, significantly outperforming both MLP baselines

Table 7: Kupiec VaR coverage rejection rates averaged over 30 seeds. Lower values indicate better calibration. p-values are from paired tests with RG-ResMoE as the reference. n.s. denotes not significant.

Slice	RG-ResMoE	MLP-L	MLP-S
5% VaR, all dates	**26.7%**	30.3% ($p<10^{-4}$)	32.7% (p=.0007)
5% VaR, high-vol	**11.9%**	14.8% ($p<10^{-4}$)	16.9% (p=.0001)
5% VaR, low-vol	**22.9%**	24.7% (p=.003)	28.2% (p=.002)
1% VaR, all dates	**9.1%**	9.9% (p=.001)	11.4% (p=.0001)
1% VaR, high-vol	**6.2%**	7.1% ($p<10^{-4}$)	7.2% ($p<10^{-5}$)
1% VaR, low-vol	5.8%	5.8% (n.s.)	6.8% (p=.053, n.s.)

Table 8: RG-ResMoE's IC advantage over MLP-L across market conditions. Relative gain is the ratio of each subset's ΔIC to the full-sample ΔIC.

Slice	Days	ΔIC ↑	Rel. gain	p
Full sample	1,890	+0.0048	ref.	$< 10^{-4}$
Top market-vol decile	189	+0.0207	4.3×	$< 10^{-4}$
COVID crisis (2020-02–06)	104	+0.0322	6.7×	$< 10^{-4}$
2022 bear market	251	+0.0079	1.6×	$< 10^{-4}$
Regime-flip windows (±10d)	826	+0.0031	0.6×	$< 10^{-4}$

Table 9: Replication on the Japanese TSE Prime market.

Model	IC ↑	RMSE ↓	QLIKE ↓	Collapse ↓
MLP-L	0.4815 ± 0.0036	0.1851 ± 0.0007	11.6 ± 34.2	6/30
MLP-L (+z)	0.4815 ± 0.0026	0.1901 ± 0.0025	41,256 ± 40,296	28/30
MoE	0.4805 ± 0.0029	0.1862 ± 0.0012	3,081 ± 7,227	22/30
RG-ResMoE	**0.4858 ± 0.0024**	**0.1846 ± 0.0004**	**0.80 ± 0.33**	**1/30**

in five of the six regime–VaR combinations. The largest improvements occur during high-volatility periods, when accurate tail-risk estimation is most critical for risk management. The only exception is the 1% low-volatility slice, where all models perform similarly and the differences are not statistically significant.

Performance under market stress. We finally examine where RG-ResMoE's ranking advantage is concentrated, particularly during periods of elevated market volatility. For each market condition, Table 8 computes RG-ResMoE's IC advantage over MLP-L using only the dates in that subset.

The gain is not uniform over time. RG-ResMoE outperforms MLP-L by +0.0048 IC in the full sample, but by +0.0207 IC in the highest-volatility decile and +0.0322 IC during the COVID crisis, corresponding to approximately 4.3× and 6.7× the full-sample gain. The 2022 bear market also amplifies the advantage, although less strongly. By contrast, regime-transition windows do not show a similar amplification. These results suggest that the benefit of residual routing is greatest during sustained periods of elevated market volatility rather than during regime transitions themselves.

8 Cross-Market Replication

We evaluate generalization beyond the U.S. market using the Japanese TSE Prime panel. Table 9 reports the core comparison under the same features, walk-forward protocol, hyperparameter selection rules, and thirty random seeds. RG-ResMoE outperforms MLP-L by +0.0043 IC (IC and RMSE $p < 10^{-4}$; QLIKE $p = 0.002$). Input concatenation is again unstable. It leaves IC unchanged while worsening RMSE and collapsing in 28 of 30 MLP-L ($+z$) seeds. The same stability ordering is reproduced on the Japanese market, suggesting that the benefit of residual routing is not market-specific.

9 Related Work

Regime switching in finance. Markov-switching models made regime dependence central to macroeconomic and volatility modeling [7–9]. Recent neural forecasting methods extend this idea through regime-aware architectures and expert routing [2, 10, 22]. These approaches demonstrate the potential value of incorporating market regimes, but differences in regime representation, routing strategy, and model capacity can complicate the attribution of performance gains. As a result, the contribution of the integration point remains unclear.

Mixture-of-experts. MoE models originated with Jacobs et al. [12] and have become a standard architecture for scalable conditional computation [6, 19], including recent time-series foundation models [15, 20]. Modern sparse MoE architectures primarily use hard top-k routing to increase model capacity while keeping the computation per input nearly constant, with expert specialization emerging through conditional activation. Soft expert combinations have also been studied to improve optimization and training stability [18]. However, most of this literature treats routing as a mechanism for computational scaling rather than as a modeling choice for incorporating market state information.

Residual learning and stable initialization. Residual networks improve optimization by learning corrections rather than complete transformations [11]. Zero- or near-zero-initialized residual branches further stabilize training [23], while warm-starting methods show that remaining close to a stable initialization can improve generalization [1]. Although these principles are well established in deep learning, their combination with gated expert routing has received comparatively little attention, particularly in financial forecasting.

10 Conclusion and Limitations

This paper studied how regime information should be incorporated into a neural network for volatility forecasting. The answer is not simply to add regime variables or adopt an MoE architecture. In our setting, directly concatenating regime variables to the forecasting input degrades performance and training stability, whereas incorporating them through residual expert routing preserves predictive accuracy and training stability. Soft residual correction on top of a frozen base forecast consistently outperforms hard routing, richer hand-crafted regime definitions, additional experts, and a standard MoE without a frozen base.

The broader implication is that nonstationary market information is safer to introduce by modulating the forecasting process than by directly determining the forecast itself. More generally, the results suggest that the value of MoE in compact forecasting models lies less in increasing model capacity than in controlling how auxiliary state information influences the prediction.

This study is limited to volatility forecasting in two equity markets using a compact MLP backbone. Whether the same design principle extends to other financial forecasting tasks, larger sequence models, or alternative forms of market state remains an important direction for future work.

References

[1] Jordan T Ash and Ryan P Adams. 2020. On warm-starting neural network training. *NeurIPS* 33 (2020), 3884–3894.

[2] Melike Bildirici and Özgür Ersin. 2014. Modeling Markov switching ARMA-GARCH neural networks models and an application to forecasting stock returns. *The Scientific World Journal* 2014, 1 (2014), 497941.

[3] Tim Bollerslev. 1986. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics* 31, 3 (1986), 307–327.

[4] Fulvio Corsi. 2009. A simple approximate long-memory model of realized volatility. *Journal of Financial Econometrics* 7, 2 (2009), 174–196.

[5] Francis X Diebold and Roberto S Mariano. 1995. Comparing predictive accuracy. *Journal of Business and Economic Statistics* 13, 3 (1995), 253–263.

[6] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[7] Stephen F Gray. 1996. Modeling the conditional distribution of interest rates as a regime-switching process. *Journal of financial economics* 42, 1 (1996), 27–62.

[8] James D Hamilton. 1989. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica: Journal of the econometric society* (1989), 357–384.

[9] James D Hamilton and Raul Susmel. 1994. Autoregressive conditional heteroskedasticity and changes in regime. *Journal of Econometrics* 64, 1-2 (1994), 307–333.

[10] Cheng He, Zhenyu Guan, Xijie Liang, Defu Lian, Jiajia Li, Enhong Chen, Patrick PC Lee, Geng Hu, and Zehao Chen. 2026. RAVEN: A Regime-Aware Variable-context Expert Network for Financial Time Series Forecasting. *arXiv preprint arXiv:2606.24062* (2026).

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[12] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation* 3, 1 (1991), 79–87.

[13] Paul H. Kupiec. 1995. Techniques for Verifying the Accuracy of Risk Measurement Models. *The Journal of Derivatives* 3, 2 (1995), 73–84. doi:10.3905/jod.1995.407942

[14] Sangheon Lee and Poongjin Cho. 2025. Graph-based stock volatility forecasting with effective transfer entropy and Hurst-based regime adaptation. *Fractal and Fractional* 9, 6 (2025), 339.

[15] Xu Liu, Juncheng Liu, Gerald Woo, Taha Aksu, Yuxuan Liang, Roger Zimmermann, Chenghao Liu, Junnan Li, Silvio Savarese, Caiming Xiong, et al. 2025. Moirai-MoE: Empowering Time Series Foundation Models with Sparse Mixture of Experts. In *International Conference on Machine Learning*. PMLR, 38940–38962.

[16] Whitney K Newey and Kenneth D West. 1987. A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica: Journal of the Econometric Society* (1987), 703–708.

[17] Andrew J Patton. 2011. Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics* 160, 1 (2011), 246–256.

[18] Joan Puigcerver, Carlos Riquelme Ruiz, Basil Mustafa, and Neil Houlsby. 2024. From sparse to soft mixtures of experts. In *International Conference on Learning Representations*, Vol. 2024. 28435–28445.

[19] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).

[20] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. 2025. Time-MoE: Billion-Scale Time Series Foundation Models with Mixture of Experts. In *International Conference on Learning Representations*. https://openreview.net/forum?id=e1wDDFmlVu

[21] Chunxia Tian, Roengchai Tansuchat, and Songsak Sriboonchitta. 2026. Regime-Aware Stock Index Forecasting Under Latent Market States: A Hybrid Statistical Learning Framework with Cross-Market Validation. *Forecasting* 8, 3 (2026), 50.

[22] Zhaojian Yu, Yinghao Wu, Genesis Wang, and Heming Weng. 2024. MIGA: Mixture-of-Experts with Group Aggregation for Stock Market Prediction. *arXiv preprint arXiv:2410.02241* (2024).

[23] Hongyi Zhang, Yann N Dauphin, and Tengyu Ma. 2019. Fixup initialization: Residual learning without normalization. *arXiv preprint arXiv:1901.09321* (2019).